

1. Applications of Linked Lists

Linked lists are widely used in various applications due to their dynamic nature and efficient memory management. Here are some key applications:

- **Data Structure Implementations:** Used to implement stacks and queues.
 - **Memory Management:** Helps in dynamic memory allocation.
 - **Data Manipulation:** Useful for representing polynomials, large numbers, and sparse matrices.
 - **Real-World Applications:** Used in image viewers, web browsers, music players, and GPS navigation.
 - **Advanced Applications:** Task scheduling, undo/redo functionality, and symbol tables in compilers.
-

2. Types of Linked Lists

1. Singly Linked List

- **Definition:** Each node contains data and a pointer to the next node.
- **Traversal:** One-way (forward).
- **Time Complexity:** $O(n)$ for searching, $O(1)$ for insertion/deletion at the head.

2. Doubly Linked List

- **Definition:** Each node contains data, a pointer to the next node, and a pointer to the previous node.
- **Traversal:** Two-way (forward and backward).
- **Time Complexity:** $O(n)$ for searching, $O(1)$ for insertion/deletion at the head or tail.

3. Circular Linked List

- **Definition:** The last node points back to the first node, forming a circular structure.
 - **Traversal:** Can start at any node and traverse in a loop.
 - **Time Complexity:** $O(n)$ for searching, $O(1)$ for insertion at the head.
-

3. Linked List vs Array Comparison

Feature	Array	Linked List
Memory Allocation	Contiguous	Non-contiguous
Size	Fixed	Dynamic
Access Time	$O(1)$	$O(n)$
Insertion/Deletion Time	$O(n)$	$O(1)$ ($O(n)$ to find position)
Space Efficiency	Can waste space	More efficient for unknown size
Structure	Single field	Nodes with data and address

Conclusion: Use arrays when the number of elements is known and fast access is required. Use linked lists for frequent insertions/deletions and when the number of elements is dynamic.

4. Java Implementations

Circular Queue using Linked List

Here is a Java implementation of a circular queue using a linked list:

```
java
VerifyOpen In EditorRunCopy code
1class Node {
2    int data; // Data to be stored in the node
3    Node next; // Pointer to the next node
4
5    Node(int newData) {
6        data = newData;
7        next = null; // Initialize next as null
8    }
9}
10
11class Queue {
12    Node front, rear; // Pointers to the front and rear of the queue
13
14    Queue() {
15        front = rear = null; // Initialize front and rear as null
16    }
17
18    // Function to insert an element into the circular queue
19    void enqueue(int value) {
20        Node newNode = new Node(value); // Create a new node
21
22        if (front == null) { // If the queue is empty
23            front = rear = newNode; // Both front and rear point to the new
node
24            rear.next = front; // Point rear's next to front to make it
circular
25        } else {
26            rear.next = newNode; // Link the new node at the end of the queue
27            rear = newNode; // Update rear to the new node
28            rear.next = front; // Point rear's next to front to maintain
circularity
29        }
30    }
31
32    // Function to delete an element from the circular queue
33    int dequeue() {
34        if (front == null) { // If the queue is empty
35            System.out.println("Queue is empty");
36            return -1; // Return -1 to indicate the queue is empty
37        }
38
39        int value = front.data; // Get the front value
40        if (front == rear) { // If there's only one node
41            front = rear = null; // Reset front and rear to null
42        } else {
43            front = front.next; // Move front to the next node
44            rear.next = front; // Update rear's next to the new front
45        }
46        return value; // Return the dequeued value
47    }
48
49    // Function to return the front value
50    int getFront() {
51        if (front == null) {
52            System.out.println("Queue is empty");
```

```

53         return -1; // Return -1 if the queue is empty
54     }
55     return front.data; // Return the front value
56 }
57
58 // Function to return the rear value
59 int getRear() {
60     if (rear == null) {
61         System.out.println("Queue is empty");
62         return -1; // Return -1 if the queue is empty
63     }
64     return rear.data; // Return the rear value
65 }
66
67 // Function to print the queue elements
68 void printQueue() {
69     if (front == null) {
70         System.out.println("Queue is empty");
71         return; // Exit if the queue is empty
72     }
73
74     Node curr = front; // Start from the front
75     do {
76         System.out.print(curr.data + " "); // Print current node's data
77         curr = curr.next; // Move to the next node
78     } while (curr != front); // Continue until we circle back to front
79     System.out.println();
80 }
81
82 public static void main(String[] args) {
83     Queue q = new Queue(); // Create a new queue
84
85     // Enqueue elements
86     q.enqueue(10);
87     q.enqueue(20);
88     q.enqueue(30);
89     q.enqueue(40);
90
91     // Print the queue
92     System.out.println("Queue elements:");
93     q.printQueue();
94
95     // Get front and rear values
96     System.out.println("Front value: " + q.getFront());
97     System.out.println("Rear value: " + q.getRear());
98
99     // Dequeue elements
100    System.out.println("Dequeued: " + q.dequeue());
101    System.out.println("Dequeued: " + q.dequeue());
102
103    // Print the queue after dequeue operations
104    System.out.println("Queue elements after dequeue:");
105    q.printQueue();
106 }
107}

```

Explanation of the Code

- **Node Class:** Represents each element in the queue.
- **Queue Class:** Manages the circular queue operations, including enqueue, dequeue, and printing the queue.

This implementation efficiently handles the circular nature of the queue using a linked list, allowing for dynamic memory usage without the limitations of a fixed-size array.

If you have any further questions or need additional implementations, feel free to ask! ### Stack Implementation using Linked List in Java

In addition to the circular queue, we can also implement a stack using a linked list. A stack follows the Last In First Out (LIFO) principle, where the last element added is the first one to be removed. Below is a detailed implementation of a stack using a singly linked list.

```
java
VerifyOpen In EditorRunCopy code
1class StackNode {
2    int data; // Data to be stored in the node
3    StackNode next; // Pointer to the next node
4
5    StackNode(int newData) {
6        data = newData;
7        next = null; // Initialize next as null
8    }
9}
10
11class Stack {
12    StackNode top; // Pointer to the top of the stack
13
14    Stack() {
15        top = null; // Initialize top as null
16    }
17
18    // Function to push an element onto the stack
19    void push(int value) {
20        StackNode newNode = new StackNode(value); // Create a new node
21        newNode.next = top; // Link the new node to the previous top
22        top = newNode; // Update top to the new node
23    }
24
25    // Function to pop an element from the stack
26    int pop() {
27        if (top == null) { // If the stack is empty
28            System.out.println("Stack is empty");
29            return -1; // Return -1 to indicate the stack is empty
30        }
31
32        int value = top.data; // Get the top value
33        top = top.next; // Move top to the next node
34        return value; // Return the popped value
35    }
36
37    // Function to return the top value without removing it
38    int peek() {
39        if (top == null) {
40            System.out.println("Stack is empty");
41            return -1; // Return -1 if the stack is empty
42        }
43        return top.data; // Return the top value
44    }
45
46    // Function to check if the stack is empty
47    boolean isEmpty() {
48        return top == null; // Return true if top is null
49    }
50}
```

```

51 // Function to print the stack elements
52 void printStack() {
53     if (top == null) {
54         System.out.println("Stack is empty");
55         return; // Exit if the stack is empty
56     }
57
58     StackNode curr = top; // Start from the top
59     while (curr != null) { // Traverse until the end
60         System.out.print(curr.data + " "); // Print current node's data
61         curr = curr.next; // Move to the next node
62     }
63     System.out.println();
64 }
65
66 public static void main(String[] args) {
67     Stack stack = new Stack(); // Create a new stack
68
69     // Push elements onto the stack
70     stack.push(10);
71     stack.push(20);
72     stack.push(30);
73     stack.push(40);
74
75     // Print the stack
76     System.out.println("Stack elements:");
77     stack.printStack();
78
79     // Peek at the top value
80     System.out.println("Top value: " + stack.peek());
81
82     // Pop elements from the stack
83     System.out.println("Popped: " + stack.pop());
84     System.out.println("Popped: " + stack.pop());
85
86     // Print the stack after pop operations
87     System.out.println("Stack elements after pop:");
88     stack.printStack();
89 }
90}

```

Explanation of the Code

- **StackNode Class:** Represents each element in the stack with a `data` field and a `next` pointer.
- **Stack Class:** Manages the stack operations:
 - **push(int value):** Adds a new node to the top of the stack.
 - **pop():** Removes a node from the top of the stack and returns its value.
 - **peek():** Returns the value of the top node without removing it.
 - **isEmpty():** Checks if the stack is empty.
 - **printStack():** Prints all elements in the stack from top to bottom.

Usage

- The `main` method demonstrates how to use the `Stack` class by pushing and popping elements, as well as printing the current state of the stack.

This implementation efficiently handles stack operations using a linked list, allowing for dynamic memory usage without the limitations of a fixed-size array.